

The Calibration Objective

Vega-normalized residuals, observation weighting, bid–ask bands, and fit quality

Vol-Fitter Technical Notes, No. 07

Vol-Fitter

July 27, 2026

Abstract

Every model in the Vol-Fitter — LQD, SVI, Multi-Core Sigmoid (MCS), local volatility — is calibrated by minimizing the *same* kind of objective, and this note documents that shared machinery so the model notes need not repeat it. Three ingredients: a residual *scale* (vega-normalized price, so the loss behaves like a volatility error while every iterate stays a valid price); a per-quote *weight* (equal, or the density-corrected time-value weighting that prevents a dense strike cluster from dominating a sparse but informative wing); and a *target* (the mid, or a bid–ask/haircut band that lets the fit sit anywhere inside the quoted spread and only resists leaving it). We give the weighting formula and its uniform-grid sanity check, the squared-hinge band objective with its small mid anchor and its space-agnostic design, and the goodness-of-fit metric that reports the *actual* calibrated misfit rather than a decorative RMS. Figures generated by the production calibrator show the density correction re-balancing weight from the ATM cluster toward the wings, and a band fit relaxing the pull toward noisy mids. Before any of it, the *quarantine* decides what enters the objective at all: every screened-out quote carries a machine-readable reason rather than vanishing silently, and naming a drop deliberately never changes it — the kept set is byte-identical to the pre-quarantine pipeline. A per-slice *vega-floor incidence* count completes the honesty: where kept quotes sit below the vega floor, the vega division of the residual has stopped meaning “vol error”, and the count says so.

Contents

1	The residual scale: vega normalization	3
2	Which quotes enter at all: the quarantine	3
3	Observation weighting	4
3.1	The problem with counting quotes	4
3.2	Density-corrected time-value weights	4
4	Bid–ask bands	5
4.1	Why a band, not a mid	5
4.2	Modes and the haircut	5
5	Goodness of fit	7
6	What is original here	7

7	Traceability	7
A	Hyperparameter atlas	8
B	Reference implementation: the band residual stack	9

1 The residual scale: vega normalization

A smile is quoted in volatility but priced in price, and the two have very different scales across moneyness: a vol bp in the wings is worth a tiny fraction of a vol bp ATM in price terms. The Vol-Fitter resolves this once, for all models, by working in *vega-normalized price* residuals. For a quote at k with total variance w , the residual is

$$r = \frac{C^{\text{model}}(k) - B(k, w)}{\partial_{\sigma} B(k, w) + \eta}, \quad \partial_{\sigma} B = \varphi(d_+) \sqrt{T}, \quad (1)$$

with a small vega floor η in the far wings. The numerator is computed in the model's native, arbitrage-preserving price space (so every iterate is a valid density); the division by vega undoes the price-to-vol scale, so the optimizer sees something close to a uniform volatility error.

Heuristic

Why not just fit volatility directly? Because for the density-based models (LQD, LV) the *price* is the arbitrage-free object and the implied vol is a nonlinear read-off; fitting price keeps the feasible set convex-clean, and the vega division recovers the vol-like loss for free. For the vol-space models (SVI, MCS) the residual is already a vol error and (1) degenerates to exactly that. One residual concept, two implementations, identical units.

Invariant

1. One objective vocabulary for every model: same residual units, same weight schemes, same band construction — a mode or scheme change means the same thing on LQD, SVI, MCS and LV.
2. Weights are mean-normalized to one, so switching schemes never silently re-tunes the data-versus-regularization balance.
3. Inside the quoted band the fit pays only the small mid anchor — the band never pushes, it only stops punishing.
4. The reported RMS is the distance to the objective that was actually minimized, under the weights that were actually used.
5. No quote is dropped silently: every screen names its reason (section 2), and naming a drop never changes it — the kept and dropped sets are byte-identical to the pre-quarantine pipeline.

2 Which quotes enter at all: the quarantine

Before a residual exists, a screen decides whether the quote does. The screens themselves are quote-preparation machinery (the OTM split, tick floor and de-Americanization order live in Note 05) — what belongs to the *objective* is the contract that none of them acts silently. Every dropped quote is returned as a quarantine record with a machine-readable reason (`api/quotes.py`): `missing_or_crossed` (no two-sided market), `tick_floor` (the price quantum, not the market, sets the IV), `nonpositive_bid`, `below_intrinsic` (a side at or below intrinsic — near-zero time value, no stable IV exists), `price_bound` (at or above the static upper bound), `iv_unresolvable` (inside

the bounds but the inversion failed), and `wing` (beyond $Z_{\max}$ ATM standard deviations). A desk auditing a thin weekly can see *why* a strike is absent, and the Quality view surfaces the counts — advisory, never a readiness gate.

Two design points keep the honesty cheap. First, the intrinsic tolerance is deliberately *zero*: it matches the inversion’s strict lower bound exactly, so introducing the classification changed no fit — the kept and dropped sets are byte-identical to the pre-quarantine pipeline (“name the drop, don’t change it”). Second, a per-slice *vega-floor incidence* diagnostic counts the *kept* quotes whose Black vega sits below 10^{-3} : where that count is material (very short-dated slices), an IV-space residual is numerically meaningless — dividing a price error by a vanishing vega manufactures noise — and the vega-normalized *price* objectives of section 1 (LQD, LV) are the authoritative view. The count is the note’s own thesis applied to itself: the residual scale is a claim about units, and the diagnostic reports where the claim thins out.

3 Observation weighting

3.1 The problem with counting quotes

Each squared residual enters the loss with a weight ω_i . With equal weights the fit is driven by wherever the quotes happen to be *dense* — typically a tight ATM cluster — so an isolated but economically important wing quote barely registers. The desk wants the opposite: weight should follow the *economic* importance of a strike, not the accident of how finely it was sampled.

3.2 Density-corrected time-value weights

The Vol-Fitter offers two schemes via `weightScheme`. The default `equal` uses unit weights. The `tv_density` scheme implements

$$\omega_i = \max(\text{TV}_i, \varepsilon) \frac{s_i}{\bar{s}}, \quad (2)$$

where TV_i is the quote’s *time value* (its OTM option price, which in forward-normalized units is its whole value), and s_i is the one-dimensional Voronoi cell width of the quote in log-strike: the stretch of the strike axis closer to this quote than to any other — half the gap to each neighbour, one-sided at the ends — i.e. how much strike range this quote alone covers. The factor TV_i is the economic weight (a quote on a richly-priced option matters more); the factor $s_i/\bar{s}$ is the *density correction*, dividing out the exchange’s listing density $1/s_i$ so that the *aggregate* weight over strike space follows the time-value measure $\text{TV}(x) dx$ rather than the raw quote histogram — without it, the densely-listed ATM cluster would outvote the wings simply by quote *count*.

Proposition 1 (Uniform-grid sanity check). *On a uniform log-strike grid all s_i are equal, so $s_i/\bar{s} = 1$ and (2) reduces to $\omega_i = \text{TV}_i$ — the pure time-value weighting, with no double-counting to correct.*

A cap `DEFAULT_MAX_MULT=10` on the spacing multiplier $s_i/\bar{s}$ stops a single isolated far-wing quote from dominating. Crucially the weights are *mean-normalized to one*, so switching schemes leaves the data-versus-regularization balance unchanged: the LQD ridge, the MCS hat ridge and the LV roughness penalty are all tuned against unit-mean weights and never silently over- or under-regularize when the scheme changes.

Figure 1 shows the effect on a non-uniform strike set with a dense ATM region and sparse wings: the dense ATM quotes are individually *down*-weighted and the sparse wing quotes *up*-weighted relative to equal, re-balancing the fit toward the full strike range while still respecting that ATM options carry more time value (the net wing-to-ATM weight ratio here is 1.3).

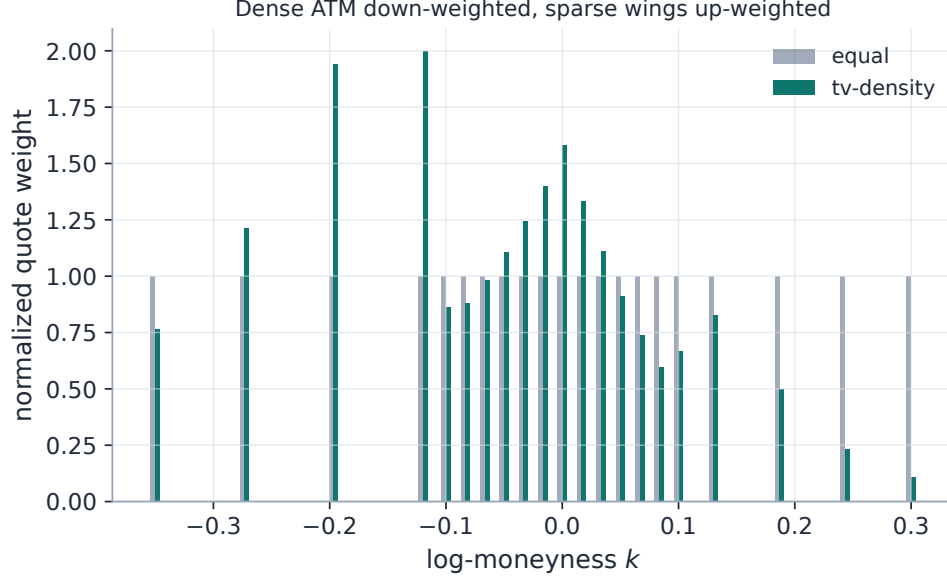


Figure 1: Equal vs density-corrected time-value weights on a non-uniform strike set. The dense ATM cluster is down-weighted per quote, the sparse wings up-weighted, so the aggregate weight follows the time-value measure rather than the quote histogram.

4 Bid-ask bands

4.1 Why a band, not a mid

A mid quote pretends a price is known exactly; a real quote is a *band* [bid, ask], and any curve inside it is consistent with the market. Fitting the mid forces the curve to chase the midpoint of every spread, including the noise in illiquid wings where the spread is wide. The band objective instead penalizes only *leaving* the band, with a gentle pull toward mid for identifiability:

$$\text{loss}_i = \max(m_i - h_i, 0)^2 + \max(l_i - m_i, 0)^2 + \theta_{\text{mid}} (m_i - \text{mid}_i)^2, \quad (3)$$

where m_i is the model value, $[l_i, h_i]$ the (possibly haircut-adjusted) band, and $\theta_{\text{mid}} = 0.05$ the mid-anchor weight. Inside the band the first two terms vanish: the fit is *free*, paying only the small anchor; once it leaves, the squared hinge pulls it back hard.

4.2 Modes and the haircut

Three modes via `fitMode`. `mid` sets $l_o = h_i = \text{mid}$ (the band collapses, recovering a mid fit). `bidask` uses the raw (bid, ask). `haircut` tightens each side toward mid by `haircut` volatility points, never crossing mid,

$$l_o = \min(\text{bid} + h, \text{mid}), \quad h_i = \max(\text{mid}, \text{ask} - h), \quad (4)$$

so a quote tighter than $2h$ collapses gracefully to a mid fit on that strike. This makes the haircut one dial that spans the whole mid-to-band spectrum: tight ATM spreads collapse onto the mid (the fit follows mids exactly where they are informative), while wide wing spreads keep a meaningful band (the fit only has to stay inside it). The hinge is monotone in the quote value, so the *same* construction works in any monotone residual space — implied vol for SVI/MCS, vega-normalized price for LQD/LV — and the band-violation subgradient sgn feeds the analytic Jacobians. Both are one line:

Listing 1: The two-sided band hinge and its subgradient (from `calib/band.py`; match production exactly).

```
1 def band_violation(model, lo, hi):    # > 0 only outside [lo, hi]; 0
    inside
2     return np.maximum(model - hi, 0.0) + np.maximum(lo - model, 0.0)
3
4 def band_violation_sign(model, lo, hi):    # +1 above, -1 below, 0
    inside
5     return np.where(model > hi, 1.0, 0.0) - np.where(model < lo, 1.0,
        0.0)
```

Figure 2 shows a band fit against a mid fit on a smile with a widening wing spread and noisy mids: the band fit is content to sit anywhere inside the shaded band, relaxing the pull toward the noisy midpoints, while the mid fit is anchored to each midpoint.

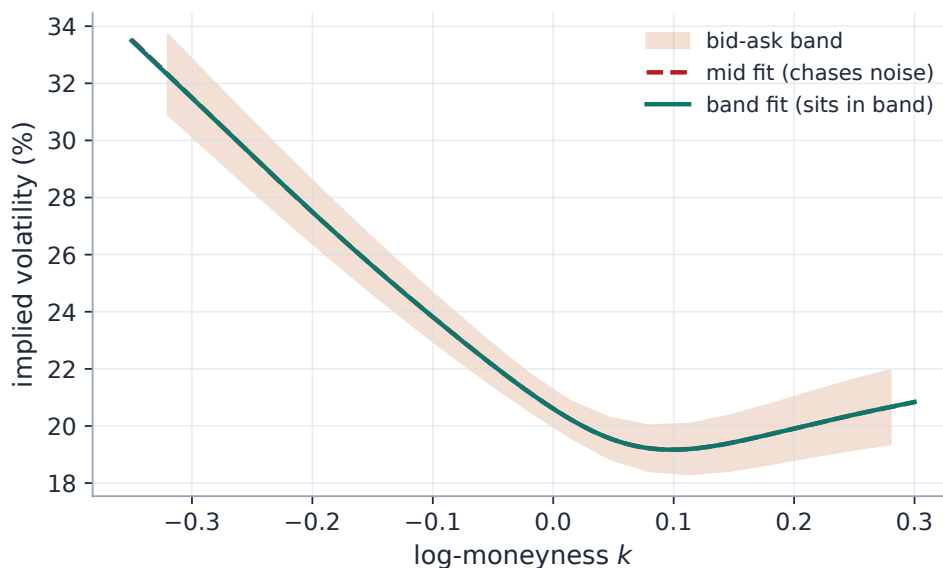


Figure 2: A bid–ask band fit (solid) vs a mid fit (dashed) with the quoted band shaded. Inside the band the fit pays only the small mid anchor, so it need not chase noisy midpoints — most valuable in wide-spread wings.

Heuristic

The band’s value scales with model flexibility. A rigid parametric model (a seven-parameter LQD) already smooths through mid noise, so the band and mid fits nearly coincide; a flexible model (a many-vertex local-vol surface) can be pulled into spurious wiggles by noisy mids, and

there the band’s freedom-to-sit-anywhere is what keeps the surface smooth. The band is a *regularizer that costs no bias*: it does not push the fit anywhere, it merely stops penalizing it for being inside the spread.

5 Goodness of fit

A reported fit error should reflect the *objective that was minimized*, not a decorative quantity. The Vol-Fitter’s weighted RMS therefore mirrors the active configuration: the distance to the fit *target* (mid, or band violation, so a fit sitting happily inside the band reports near-zero error even if it is far from mid), under the active *weighting* scheme, including the var-swap penalty term when active. It is returned as a $(\sum \omega r^2, \sum \omega)$ pair so node errors aggregate correctly into a surface RMS. This is why the same surface can report a different RMS under `mid` and `bidask`: the two minimize different objectives, and the metric is honest about which.

6 What is original here

The vega normalization is standard; the contributions are the *density-corrected* weighting (2) — separating economic importance (time value) from sampling density (Voronoi spacing), with the mean-normalization that keeps regularization invariant across schemes — and the *space-agnostic squared-hinge band* (3) whose monotone subgradient plugs into every model’s analytic Jacobian, so bid–ask fitting is a first-class mode rather than a bolt-on. Together they let the desk express “fit what matters, to within the spread” in two orthogonal knobs.

7 Traceability

Table 1: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
Density-corrected weights match (2) the design note’s worked example		<code>calib/weights.py</code>	<code>test_weights.py::</code> <code>test_doc_worked_example</code>
Uniform grid reduces to pure time value	section 3	<code>calib/weights.py</code>	<code>test_weights.py::</code> <code>test_uniform_grid_reduces_</code> <code>to_time_value</code>
Dense cluster down-weighted; (2) mean-one normalization		<code>calib/weights.py::</code> <code>resolve_weights</code>	<code>test_weights.py::test_</code> <code>dense_region_downweighted,</code> <code>::test_resolve_weights_</code> <code>equal_is_none_and_tv_is_</code> <code>mean_one</code>
The scheme moves every model	invariant 1	<code>calib/weights.py</code>	<code>test_weights.py::</code> <code>test_weight_scheme_moves_</code> <code>every_model</code>
Band modes; haircut collapses gracefully to mid	section 4	<code>calib/band.py::</code> <code>resolve_band</code>	<code>test_band_fit.py::</code> <code>test_resolve_band_modes,</code> <code>::test_haircut_default_is_</code> <code>half_vol_point</code>

Claim	Object	Code anchor	Test anchor
Zero residual inside the band	(3)	calib/band.py	test_band_fit.py:: test_band_residuals_zero_ inside_band
RMS mirrors the minimized objective	section 5	calib/rms.py	test_rms.py::test_mid_ mode_is_weighted_rms_ distance_to_mid, ::test_ band_mode_zero_inside_ band_distance_outside
Band-mode RMS never exceeds mid-mode; surface pooling	section 5	calib/rms.py	test_rms.py::test_band_ mode_rms_not_above_mid_ mode, ::test_surface_rms_ pools_expiries
Quarantine reasons named; clean chains screen nothing; quality surfaces counts advisory	section 2	api/quotes.py	test_quote_screen.py:: test_clean_chain_screens_ nothing, ::test_tick_ floor_and_crossed_reasons, ::test_quality_surfaces_ screen_counts_advisory
Vega-floor incidence counts kept low-vega quotes	section 2	api/quotes.py	test_quote_screen.py:: test_vega_floor_ diagnostic_counts_kept_ quotes

A Hyperparameter atlas

Table 2: Calibration-objective hyperparameters.

Knob	Default	Role
<i>Surfaced (FitSettings / OptionsSettings)</i>		
weightScheme	equal	equal or tv_density.
fitMode	mid	mid / bidask / haircut.
haircut h	0.005	Haircut in vol points (each band side toward mid).
midAnchorWeight θ_{mid}	0.05	Mid anchor strength vs band penalty (= 1).
<i>Hidden</i>		
DEFAULT_MAX_MULT	10	Cap on the Voronoi spacing multiplier $s_i/\bar{s}$.
ε	10^{-12}	Floor on time value / spacing.
η (vega floor)	10^{-4}	Floor on Black vega in (1).
TICK_FLOOR_TICKS	3	OTM <i>bid</i> floor in ticks on real-feed chains (screen mechanics in Note 05; the reason taxonomy here).
INTRINSIC_TOL	0	Intrinsic classification tolerance — zero by design, so naming the drop changed no fit (section 2).
VEGA_FLOOR_DIAG	10^{-3}	Per-quote vega threshold of the kept-quote incidence count (section 2).

Performance. Weighting is $O(m \log m)$ (one sort) and the band residual is $O(m)$, both negligible against the fit; the band subgradient is analytic so it adds nothing to Jacobian assembly. Switching schemes or modes bumps the options version and triggers exactly one refit.

B Reference implementation: the band residual stack

The full objective stacks two blocks per quote: the band violation (scaled by the per-quote weight, or 1/vega in the LQD/LV price space) and a weak mid anchor of strength θ_{mid} , so the fit is free inside the band yet never drifts off a one-sided quote. Squaring and summing reproduces the loss $\max(m - \text{hi}, 0)^2 + \max(\text{lo} - m, 0)^2 + \theta_{\text{mid}}(m - \text{mid})^2$. Matches production to 10^{-15} .

Listing 2: The stacked band-objective residual (distilled from `calib/band.py`).

```

1 def band_residuals(model, lo, hi, mid, scale, mid_anchor_weight):
2     viol = scale * band_violation(model, lo, hi)           # push inside
3     anchor = np.sqrt(mid_anchor_weight) * scale * (model - mid) #
4     return np.concatenate([viol, anchor])                 # least-
    squares stack (length 2N)

```

References

- [1] Vol-Fitter Technical Note, *Density-corrected time-value weights* (Docs/iv_time_value_density_weights.tex).
- [2] J. Gatheral. *The Volatility Surface*. Wiley, 2006.